

Analyzing Stress through Textual and Quantitative Data with Naive Bayes Classifier

RIJAN POKHREL¹, (Student, IOE), and SABIN ACHARYA², (Student, IOE)

¹Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: rijan.077bei034@tcioe.edu.np)

²Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: sabin.077bei035@tcioe.edu.np)

This lab was performed under the supervision of Er. Dinesh Baniya Kshetri in Department of Electronics and Computer Engineering, Thapathali Campus, IOE, T.U.

ABSTRACT In this lab report, the stress of people based on their reddit posts on different subreddits is predicted using the Naive Bayes Classifier. The dataset is visualized using bar-plots and scatter-plots to visualize if the features hold any significance in labeling the datapoints. The features with seemingly no significance in labeling the datapoints were dropped to avoid noise in the model. The text data is properly analyzed and different preprocessing like lemmatization and stemming and feature extraction techniques like word-embedding/vectorization, words/character count extraction are applied to get the appropriate features from the text data that are required by Naive Bayes Classifier. Different word-embedding techniques were applied and analyzed to find the suitable one for the dataset. Similarly, different types of Naive Bayes Classifier were used and analyzed to find the suitable classifier for the dataset. To analyze the model, different evaluation metrics like accuracy, confusion matrix, heatmap, PR curve and ROC curves are used.

INDEX TERMS Feature Engineering, Lemmatization, Naive Bayes Classifier, PR Curve, ROC Curve, Sentiment Analysis, Stemming, Tokenization, Word Cloud, Word Embedding

I. INTRODUCTION

IN data science, classification is the concept of predicting the category or class label of a given datapoint based on its features. It is a type of supervised learning which used labeled dataset to train the model and then predicts the label for test data. There are different algorithms for classification like Decision Tree Classifier, Rule Based Classifier, K-Nearest Neighbours(KNN) Classifier, Logistic Regression Classifier, Support Vector Machines(SVM), Neural Networks, Random Forest Classifier, Linear Discriminant Analysis (LDA), Naive Bayes Classifier, and many more. Among all these classifiers, Naive Bayes Classifier is based on the probabilistic statistics of the data points using the Bayes theorem.

A. LEARNING ALGORITHM CATEGORIES

Learning algorithms can be loosely categorized into 2 categories, they are: Generative approach and Discriminative approach. Both approaches can be used for supervised classification, but their approach to separate the classes is different.

1) Discriminative Approach

In the Discriminative approach, the model tries to learn and estimate the conditional probability of the class given the features i.e. $P(class|features)$.

A Discriminative models do not care about the underlying structure of the data like generative models. Instead, they are focused on finding an threshold line to separate the classes.

2) Generative Approach

Generative model are a class of ML models which are designed for generating data samples that are similar to the training data. Unlike discriminative

model that focuses on predicting a label on the basis of given input, generative models attempt to model underlying distribution of data such that it can generate new samples from the distribution. Examples: Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), and some types of deep learning architectures such as the PixelRNN.

B. DIFFERENT PHILOSOPHICAL APPROACHES TO STATISTICAL INFERENCE

1) Frequentist's View

In Frequentist's view, the probability is related to the frequency of repeated events. In this view, the probability of a coin landing heads is the proportion of times it would land on heads if the tossing was performed infinitely many times.

It assumes that there are infinite random data and a fixed model. Since it works with frequently repeated events, it measures how data varies given a fixed model. This approach works well when there is huge data and few parameters to estimate, which is improbable in the real world. For example, a deep neural network model with large data but a massive number of learning parameters.

2) Bayesian's View

In Bayesian's view, fundamental of probability is related to the certainty or uncertainty of events. If a Bayesian says that there is 50% probability of getting heads or tails, the proportion of heads or tails in infinitely many random experiments are not used. Instead, it represents the degree of certainty/-belief of event based on the knowledge about event domain using probability. The belief is updated as new evidence is observed.

It assumes that data we observed is fixed, but the parameters are random. This approach can be used on any arbitrary number of data and parameters. Bayesian's approach can be used in online learning as well, where data comes in a small portion. In this case, from each portion of our data, we can compute the posterior and later use it as a prior for another portion of data.

C. COMPARISON OF NAIVE BAYES WITH OTHER CLASSIFIERS

Naive Bayes is the most popular classification algorithm. It is faster to train and classify than more complex models like Neural Networks or SVM having large datasets. It requires less training data to achieve good performance compared to other algorithms making it suitable for limited data. It handles high dimensional data well that is beneficial for text classification tasks that outperforms other methods. It is also more interpretable than 'black box' models like deep neural networks as the impact of each feature on classification can be understood.

However, Naive Bayes can be outperformed by other algorithms in the case where there are complex interactions between the features where Random forests, Decision trees can capture this complex relationship easily. SVM generally provides best accuracy when dealing with the continuous features and can handle nonlinear decision boundaries more effectively. Deep learning models, while requiring more data and computational resources, can significantly outperform Naive Bayes on complex tasks like image recognition or natural language processing where hierarchical feature learning is beneficial.

D. PROS OF NAIVE BAYES

Naive Bayes works well with high-dimensional data like classification where the number of features can be very large. Also it works very well with the datasets having limited datapoints as it requires small amounts of training data to estimate the necessary parameters. Naive Bayes works well with the dataset having irrelevant features since during probabilities estimation they tend to cancel each other. Additionally, it provides probabilities estimates too rather than the classifications only which can be useful in many applications.

E. CONS OF NAIVE BAYES

The first and foremost drawback of this algorithm is its assumption of feature independence that is often unrealistic in the real-world. It performs poorly with the dataset having the features having strong correlation. The algorithms can provide miserable

probabilities estimates even if the classification accuracy is high.

Naive Bayes also struggles with the input features that are absent in the training data. Although it can be mitigated by the laplace smoothing, it is not ideal for capturing complex relationships in the data. Naive Bayes can handle both the categorical and continuous data, it requires binning of the continuous features that can lead to loss of the information.

II. LITERATURE REVIEW

The paper [1] tackles two major aspects and 12 major factors that has huge contributions in the machine learning literature with regard to classification. Some of the central topics of discussion are representation, evaluation, optimization, overfitting, bias, and variance. Representation examines how data are translated in a language understandable by computers i.e. the space of all possible hypotheses. The challenges come in testing the lines of complexity and generalization. All functions that score the quality of classifiers, internal and external, are not usually identical to those used for optimization. Optimization techniques are all important methods that search with different strategies for high-scoring classifiers within the space of hypotheses. Efficiency and solution quality may be significantly influenced by this component. Cross-validation, regularization, and statistical tests deal with overfitting, while these might shift the model to underfitting if not balanced. Bias and variance are two central aspects with regard to the generalization error of a model. Good models would reduce both bias and variance as much as possible for effective performance. Knowledge integration is a step that brings in domain knowledge to make learning more effective. "No free lunch" theorems imply that no one learner can be the best for all functions, but in practice, common algorithms work quite well across real-world scenarios because functions are not uniformly distributed.

The work on paper [3] focuses on the detection of the accuracy of fake news through different classification techniques. Fake news is significantly affecting social life, mainly in politics and education. They implemented a model for fake news

detection using various classifiers—SVM, Naïve Bayes, Passive Aggressive Classifier. They performed the classification on very limited datasets. The model used here uses TF-IDF for feature extraction and SVM as the classifier. The system accuracy achieved was 95.05%.

III. METHODOLOGY

A. MATHEMATICAL FOUNDATIONS

1) Bayes Theorem

One of the basic principles in probability theory and statistics, Bayes' theorem is named after the 18th-century British mathematician Thomas Bayes. It provides the means to update the probability for hypotheses upon receipt of new evidence or information. The theorem is expressed as an equation relating conditional probabilities.

Basically, Bayes' theorem explains the degree of probability of some event based on prior knowledge of conditions that might be related to the event. It is really useful when dealing with problems that involve some type of initial belief about how likely something is to happen (prior probability) and updating this belief in light of new evidence.

It is formulated using the formula of equation-(1) which helps to turn around the condition on the probabilities. It is the basis for the Naive Bayes classifier. It assumes that each feature is independent of all the other features. It is highly effective in text classification and spam filtering.

Bayes' theorem has wide uses beyond just being used for classification. It is applied in medical diagnosis for determining the probability of diseases given symptoms, in spam detection to determine how likely it is that an e-mail with certain content is spam, and in legal proceedings, updating the probability of a hypothesis upon the appearance of new evidence. The ability to include prior knowledge and to further update the probability makes it versatile and a very powerful tool in many study fields and practical applications.

2) Conditional Probability

Conditional probability refers to the likelihood of an event occurring on the ground of another event previously occurred. It helps to adjust the probability estimate due to added information. Mathematically, the conditional probability of event (A)

given event (B) is denoted as $P(A|B)$ and defined using equation-(2). This formula can be interpreted as : “Out of all the times when (B) occurs, what fraction of those times does (A) also occur?”

Conditional probability is crucial in many real-world applications, such as medical diagnosis, where a doctor has to estimate the chances of illness given certain symptoms, and weather forecasting, where a meteorologist needs to predict the weather patterns based on current conditions. It is the core concept of Naive Bayes, and many other concepts in probability and statistics. Despite the assumption of the feature independence of the naive bayes , it works well with the conditional probabilities.

3) Independence Assumption

The Independence Assumption also known as ‘naive’ is the core concept that gives the Naive Bayes functionality. This core principle simplifies the model and makes the calculations manageable when dealing with the high-dimensional data. Independence Assumption states that given the class label, the features are conditionally independent of each other. The presence of one feature should not affect the presence or absence of any other features.

Mathematically, given the target class C and features $X_1, X_2, \dots, X_n$, this assumption is given by the equation-(3).

It allows the algorithm to treat each column independently while calculating probabilities. But in real world scenarios, features are mostly dependent. For example, in text classification, the presence of the word "New" might be correlated with the presence of "York" if we're classifying articles about cities. Despite rare independent features , the Naive Bayes often performs well. It may be because of:

- 1) Even if the features are not truly independent, treating them as independent can still capture enough information for making accurate predictions.
- 2) The algorithm requires that the correct class has highest probabilities and doesn't require the exact probability estimates.
- 3) When dealing with the limited training data, it prevents overfitting by reducing the complexity of the model.

- 4) In high/multi-dimensional data space, the effect of feature dependencies often ‘averages out’ many features.

However, naive bayes can lead to suboptimal performance in the cases where feature dependencies are too strong to ignore.

4) Likelihood and prior probability

Likelihood refers to the probability of observing the given features for a particular class. Considering Independence Assumption, it is given as the product of the probabilities of each features. The likelihood gives measure of how well the observed features matches each class based on the available training data. Mathematically, for features $X_1, X_2, \dots, X_n$ and class C, the likelihood is:

$$P(X_1|C) * P(X_2|C) * \dots * P(X_n|C)$$

Prior probabilities represents the initial belief about the probabilities of each class before considering any features. It is estimated from the initial training data. The prior probabilities give the background knowledge and assumptions about the class distributions. This probabilities is useful when we have imbalanced datasets or when incorporating the domain knowledge into the model.

The likelihood gives how well the features match each class while prior gives the baseline probabilities of each target. In naive bayes algorithm both prior and likelihood are combined for calculating posterior probabilities. By combining these both together, a balanced view that considers both feature evidence and prior knowledge is obtained. This combination allows Naive Bayes to make more informed predictions, especially in cases where the feature evidence alone might be ambiguous or misleading.

5) Posterior probability

Given the observed evidence or features, Posterior Probability gives the probability of a particular class or hypothesis being true. It is calculated by combining the likelihood of the observed features given class, prior probabilities and normalizing factor called evidence. Mathematically, it is expressed using the equation-(7)

Posterior probabilities serves as the basis for classification decisions in Naive Bayes. The algorithm

selects the class with highest posterior probabilities as its prediction. This approach also gives the model's confidence in its prediction. Posterior probabilities are particularly useful in the case where the cost of misclassification varies between the classes. Additionally it also provides a way to incorporate prior knowledge or class imbalances into the classification process.

6) Maximum A Posteriori (MAP) estimation

Maximum A Posteriori (MAP) estimation is a statistical method to determine the most probable class for a given set of features. In Naive Bayes, MAP is used to select a class that has the highest posterior probability of the given observed columns. In simple words, it is an extension of maximum likelihood estimation of prior probabilities. Mathematically, MAP estimation can be expressed as in equation-(8).

MAP provides a way to make classification decisions by selecting the class having highest probabilities. Also it naturally combines the prior probabilities and likelihood of the features given each class thus balancing the evidence of the data with our prior knowledge. It is most useful when dealing with imbalanced datasets or when having strong prior beliefs of class probabilities.

MAP estimation often takes the logarithm of the probabilities for avoiding the numerical underflow with the small probabilities. It transforms the product of the probabilities into the sum of log probabilities that is computationally stable. Then the highest sum is selected as a prediction.

7) Laplace smoothing

It is also known as additive smoothing or Lidstone smoothing. It is technique to handle the zero-probability problem. It occurs when a feature-class does not occur in the prediction, resulting in the zero probabilities that can derail the entire calculations. The gist idea behind the Laplace smoothing is to add a small constant to all features to avoid all the probability to be zero. Mathematically, if we have k possible values for a feature, the smoothed probability is calculated using the formula in equation-(9)

Laplace Smoothing prevents zero probabilities,

ensuring that model can handle unseen feature values in new data. It is important for the multiplicative nature of Naive Bayes, where a single probabilities can nullify the posterior probabilities despite the probabilities of other features. Also, smoothing prevents overfitting while dealing with the small datasets or sparse features.

The choice of α can impact the model's performance. Greater value of α gives more smoothing but may introduce bias. Generally, $\alpha=1$ is taken but cross-validation can also be used to tune this parameter. Laplace Smoothing improves the robustness and generalization ability making it more reliable for real world classification tasks.

B. DATASET DESCRIPTION

The Stress Dataset consists of 2838 different textual description taken from the people's social media posts along with the different subreddits in which they were posted along with the confidence score of the label associated with the text and the time-stamp of the post. It also contains the features like post-id and sentence-range of the post from which the textual data was taken out which directly seem to hold no importance in the stress prediction. The textual description is available under 'text' column and the stress is classified as 1 for Stress and 0 for No-stress under 'label' column. The dataset consisted of categorical data for sub-reddit and confidence score, numerical data for social-timestamp and textual data for text column.

C. SYSTEM METHODOLOGY

The System Block Diagram of the Methodology used for classification problem is shown in figure-(1). The steps used for classification are described as follows:

D. DATA ANALYSIS AND VISUALIZATION

Upon importing the dataset the first step is to visualize how the dataset quality is and how are the dataset features essential in classifying a datapoint.

1) Checking Null Values

The dataset is first checked if it consists of any null values. This dataset didn't consist of any null values which can be seen from the table-(1). So, it requires no effort in handling the null dataset values.

2) Stress Visualization based on Different Time Parameters and Attributes using Bar chart

From the timestamp of the post, different time informations like the month, month-days, week-days in which the post was posted in reddit. These informations are extracted as shown in code snippet-(1). From these time information, the occurrence of stress or non-stress posts in different times frames like week-days, month-days or months can be analyzed. With these information the distribution of stress posts among these time-frames can be visualized which can be analyze if the timestamp information serve any important purpose in classifying the datapoints.

The bar chart with count of stress and non-stress posts with respect to these time-frames are plotted and visualized using the code shown in code snippet-(2). Using Similar Approach, the bar-chart of the distribution of stress is also plotted and visualized for subreddits attribute.

3) Data Visualization using Scatterplot

Also the data was visualized using the scatterplot of subreddits vs confidence to see the distribution of stress along these categories. Since both these attributes are of categorical type, it created overlapping scatterplot. To mitigate the overlapping a small offset is added to each point of in the confidence score axis to visualize the data points scatter properly.

4) Word Cloud Visualization

Word Cloud is also known as tag cloud or text cloud. It is a popular visual representation of textual data which displays the words in varying sizes and colors based on its frequency or importance within a given text or corpus. This visualization technique offers a pleasing way to quickly grasp most prominent keywords in a large body of text. The arrangement of the words in the cloud can be random or have specific patterns such as alphabetical order or custom shapes that relate the content's theme. The preprocessing steps like stop word removal are done to focus only on the meaningful words in the corpus.

Words cloud does not convey context, sentiment or relationships between words. It also tends to

emphasize individual words rather than the phrases that might be expressed in multiple words. In this dataset, the word cloud is used to visualize the words that occur most frequently in the texts of stress posts and non-stress posts separately. It is implemented as shown in the code-snippet-(3).

E. FEATURE ENGINEERING

Feature engineering is the process of creating new features or making changes in existing features to enhance model performance in machine learning and data analysis. In the domain of natural language processing and text analytics, feature engineering is changing raw text into forms that can be effectively processed by machine learning algorithms is very important. Although often done with basic text preprocessing like tokenization, lowercasing and removal of punctuation, the actual Operations extend way beyond the basics. First, more complex feature engineering in text analysis will include the following: n-grams: which allow capturing phrasal information, TF-IDF: to weight word importance and different linguistic techniques that can extract structural and semantic information from text.

1) Adding Word-count and Character-count Features

In this dataset, two additional features are added: word count and character count. These features were incorporated to explore whether the length of text inputs correlates with our target label(stressed or not stressed). After incorporating them, an analysis is done to assess any potential relationships between the number of words or characters and the output labels. To ensure consistency and comparability across different scales, min-max scaling is done to normalize it.

2) Stop Words and Punctuation Removal

Stop Words are the most frequent words in language and have little lexical content or minimal significance in conveying meaning of the text. These words are words that are filtered out or ignored during the processing of the textual data. Examples of stopwords in English include articles (a, an, the), prepositions (in, on, at), conjunctions (and, but, or), and pronouns (I, you, he, she, it) etc.

By removing the stopwords, the algorithms can focus on the more meaningful words that have high significance in the text. Based on the specific application, language or domain list of stopwords varies. For instance, in a technical document analysis, words like "figure" or "table" might be considered stopwords, whereas they wouldn't be in general text analysis.

Although this reduces the dimensionality of the text data to a great extent and improves efficiency in processing, stopword removal generally causes loss of some contextual information. Therefore, with regards to an NLP task at hand, a careful decision should be made as to whether or not to remove the stopwords, and the words that are to be added into the stopwords list.

The stopwords are obtained from the *stopwords* module of the python. Similarly, the punctuations are obtained from the *string* module. The code for stopwords and punctuation removal is shown in code-snippet-(4).

3) Stemming and Lemmatization

Stemming and Lemmatization are the process to remove the inflection from the word and reduce them to the root words. Languages tend to have various inflected words for example 'feel', 'feels', 'felt' and 'feeling' all refer to the same meaning. But presence of all these inflected words will be treated different in the process of word embedding and vectorization which increases the dimensions of the dataset. Reducing them to root words will preserve their importance while reducing the dimensions. The two basic processes to remove these inflections are:

a: Stemming

It is a basic text preprocessing technique for handling the textual data. It reduces words to their root or base form known as stem. Its core goal is to normalize the words by removing suffixes, prefixes or infixes and grouping the words with similar core meaning together. This technique improves the performance of sentiment analysis, text classification by reducing the vocabulary size of the text corpus.

Porter's algorithm is the most popular stemming algorithm used in English. Stemming sometimes

produces stems that are not actual words or can over stem leading to loss of meaning.

b: Lemmatization

It is an advanced text normalization technique that reduces words to its base or dictionary form also known as lemma. Based on the word's part of speech and context within the sentence it uses vocabulary and morphological analysis for identifying the correct lemma. It is usually more accurate than stemming but requires more computational power. It is especially used in machine translation or text summarization where preserving the exact meaning of the words is crucial. It generally uses part-of-speech tagging as part of its process that increases processing time and complexity.

Hence lemmatization that reduces words to its dictionary base is generally preferred. The choice between stemming and lemmatization depends on requirements of the textual data and its corresponding accuracy.

For this dataset, both of these techniques are applied and the accuracy of the model is observed to select the best technique for the dataset.

4) Word Embedding

It is the technique used to generate a vector of real numbers from the words or phrases in the text-corpus. It is done because the machine learning algorithms tend to fail to work with raw data while work well with numerical data. One of the common approaches for vectorizing is the Bag-of-Words(BoW) approach. In this approach, a document is represented as a vector where each dimension corresponds to a word in vocabulary, and the value represents the frequency of the word in the document. BoW does not save the word order and semantic relationships between the words. The improvement over BoW is the Term Frequency-Inverse Document Frequency (TF-IDF) method. This method not only considers the frequency of the words in the document but also weighs them by its significance across the entire corpus. It gives the higher weight to words that are frequent in the particular document but are rare across all the other documents, thus capturing more meaningful features to the text.

More advanced techniques for word embeddings are Word2Vec, GloVe, or FastText. It creates dense vector of words which captures semantic relationships allowing similar words to have similar vector representations. This embedding creates document vectors by averaging or combining the vectors of individual words in the document. The generation of numerical vector highly effects the model performance so appropriate approach should be used to generate them.

a: Tokenization

It is a mandatory step for transforming raw text into structured data that can be analyzed by ML algorithms. It is the process that breaks down text into smaller units called tokens that can be characters, sub words or phrases depending on the specific requirements of the task. The core goal of tokenization is to create a meaningful representation of text that preserves semantic information simplifying the computational processing.

The most basic method for tokenization is white space tokenization that simply splits text on spaces and punctuation. Other complex methods are rule based tokenization that employs language specific rules for handling complex cases like contractions and hyphenated words, regular expression tokenization that uses pattern matching for identifying token boundaries. Also for handling out-of-vocabulary words and morphologically rich languages advanced techniques such as subword tokenization (Byte Pair Encoding, Wordpiece) that breaks words into smaller units are used.

Tokenization has several challenges while dealing with diverse languages and domains. Some of the challenges are: Handling Contractions, Hyphenated Words and multi word expressions as well as correctly interpreting punctuation and special characters. Also domain specific terminology and abbreviations also challenges the tokenization processes.

Tokenization is the basis for feature extraction that directly influences vocabulary size and sparsity of the data. The quality of tokenization affects the performance of the sentiment analysis, text classification, machine translation etc.

For this dataset, word-based tokenization is used.

Some most popular methods for word-embedding are as follows:

b: TDF-IDF Vectorization

It is a statistical measure for evaluating the significance of the words in a document relative to corpus. This techniques combines two key components:

- 1) **TF (Term Frequency)**: It measures how often a term appears in the document. The TF is calculated by dividing the number of times a term appears in a document by the total number of terms in that document as in equation-(4).
- 2) **IDF (Inverse Document Frequency)**: It measures the term's rareness across the entire corpus. It basically gives how important a term is across the entire corpus. The IDF is computed as the logarithm of the total number of documents divided by the number of documents containing the term as in equation-(5).

The final TF-IDF score is then calculated by multiplying TF and IDF values as in equation-(6).

The TF-IDF scores give insights into the importance of the words within the documents. A high TF-IDF score means that the word is very frequent in the specific document and is rare across the corpus thus signifying that it holds particular importance or is characteristic of that document's content. Also, common words like articles, prepositions have low TF-IDF scores due to their high frequency across the documents.

Hence, TD-IDF is very effective in various scenarios but it doesn't captures any word order or semantic relationships between the words.

c: Hashing vectorization

It is an efficient technique for converting collection of text documents into a fixed size vector representation. It is particularly useful where dealing with the large vocabularies and high-dimensional features spaces is computationally inefficient.

The main idea behind hashing is to map words or n-grams directly to indices in a vector thus avoiding the need for maintaining a vocabulary. It thus allows fixed size feature vector regardless of the vocabulary size that is beneficial while dealing with streaming data where new words are continuously

encountered. Also it is memory efficient since it does not require large vocabulary for index mapping. The potential drawback of this vectorization is the possibility of hash collisions where different words map to same index. It can lead to loss of information.

The process of hashing vectorization typically works as follows: For each token (word or n-gram) in a document, a hash function is applied to generate a hash value. This hash value is then modulo divided by the desired dimensionality of the feature vector to determine the index where the token's count will be incremented. The result is a fixed-length vector where each dimension represents the count of tokens that hash to that particular index.

It's widely used in applications like spam detection, ad click prediction, and large-scale text classification. Despite being computationally efficient, it sacrifices some interpretability as it's not possible to reverse the process and determine which words contributed to which features in the resulting vector.

d: Count Vectorization

This vectorization is used to convert a collection of text documents into a matrix of token counts. It creates a numerical representation of text data which can be used as input for various algorithms.

The steps for this vectorization is : Firstly, the text is tokenized that breaks it down into individual words or n-grams. Then, a vocabulary is formed from all unique tokens across the corpus. Each document is then represented as a vector where each element corresponds to a token in the vocabulary. The value of each element is the count of how many times that token appears in the document.

The main advantage of count vectorization is that the resulting matrix shows which words appear in each document with its frequency. It makes it particularly useful for tasks like document classification where the presence or absence of certain words matters most.

It also has some limitations. It doesn't capture the order of words or their context within the document, which can be crucial for understanding meaning. Additionally, it often results in high-dimensional, sparse matrices, especially when

dealing with large vocabularies. This can lead to computational challenges and may necessitate dimensionality reduction techniques.

Count Vectorization is often used in conjunction with other preprocessing techniques like stopwords removal, stemming, or lemmatization to improve its effectiveness.

For this dataset, all these vectorization methods are tested and the one which gives the highest model accuracy is selected.

F. ONE-HOT ENCODING OF CATEGORICAL DATA

One-hot encoding is a standard way through which categorical data are transformed to numerical to feed to machine learning algorithms. With that, this method treats each category of the categorical feature as if it were a different binary column. The number of categories is the number of columns. Each category has '1' if the instance falls in that particular category and '0' otherwise. One-hot encoding is very important in removing the ordinal relationship between the categories so that algorithms treat every category equally.

One-hot encoding was done on the subreddit columns to ensure that these categorical variables were in a form that could be used by machine learning models. It is particularly useful in case of nominal data, since this encoding enables the model to capture a presence or absence for a category. One-hot encoding at this step also ensures that our dataset is well-prepared for a variety of predictive modeling tasks especially for Naive Bayes in this report.

G. MODEL TRAINING

The following steps were performed to train the model on the above prepared dataset.

1) Label and Feature Separation

After analyzing all the columns' correlation with the output column the unnecessary columns are dropped. The columns timestamp, sentence-range, post-id are dropped accordingly. Additionally, the vectorized text data are added to the dataframe while removing the raw texts, lemmatized texts and stemmed-texts. After that the features were separated into a separate variable X and the labels into a variable Y.

2) Train-Test Split

After all the data preprocessing, the data is splitted into training and test data in the ratio of 80:20. It separated 2270 data into training set and 568 datapoints into the test set.

3) Selection of Naive Bayes Type

Naive Bayes are of different types and perform well on different types of datasets. Some of them are:

a: Gaussian Naive bayes

It is specially designed for continuous data. It assumes that the data follows Normal distribution. It is suitable for datasets having continuous variables such as measurements or numerical attributes. The likelihood of the features is estimated by calculating mean and standard deviation of the training data. It is specially used in applications like medical diagnosis or weather prediction where data are mostly numerical.

b: Multinomial Naive Bayes

It is used for discrete data. It is especially suited for classification tasks. It is appropriate for the features representing count or frequencies i.e. multinomial distribution. It calculates the probability of each class by multiplying probabilities of each feature given that class. Multinomial NB is widely used in natural language processing tasks like spam detection, sentiment analysis, and document categorization. It is highly efficient in handling the high-dimensional data typically in text processing.

c: Hybrid Naive Bayes

It is also known as Mixed Naive Bayes. It combines different types of Naive Bayes models for handling datasets with the mixed feature types. It is widely useful while handling both the categorical and numerical features. It applies the appropriate probability distribution for each feature type - typically using Gaussian distribution for continuous features and multinomial or Bernoulli distributions for categorical features.

Model from all these types are trained one by one and the one with highest accuracy for this dataset is selected.

4) Model Training on Test Data

After all the necessary preprocessing and model selection, the model is trained. The model is imported from the scikit learn library and trained on the splitted 80% data of the whole dataset.

5) Model Optimization via Cross Validation

To find the best hyperparameter for the best accuracy, cross validation is done. Grid search cv was done with the parameters as $\alpha = 0.1, 0.2, 0.3, 0.05, 0.7$ for Multinomial Naive Bayes.

H. MODEL EVALUATION

Different evaluation metrics like accuracy, classification report, confusion matrix, heatmap, true positive rates, false positive rates, ROC curve, PR curve are used to evaluate the model based on its prediction on the test dataset.

1) Class Prediction with Test Data

The 20% of the data separated as the test data which are completely unknown to the model are fed to the model to predict their labels for each of the models.

2) Evaluation By Confusion Matrix and Heatmap

The confusion matrix is generated using the metrics module of the sklearn library. It shows the true positive values, true negative values, false positive values and false negative value counts which can be used to generate different insights about the model. To make the visualization easier, the confusion matrix is visualized using the heatmap from the seaborn library.

3) Evaluation By Accuracy and Classification Report

The performance of the Naive Bayes models are evaluated using accuracy and classification report. The accuracy metric provides an overall measure of the model's correctness, while the classification report gives insights into precision, recall, and F1-score for each class. These metrics are generated using the metrics module of the sklearn library.

4) Evaluation By ROC Curve and AUC

ROC is a powerful tool for evaluating the performance of binary classification models. It plots

the TPR(sensitivity or recall) against the FPR(1-specificity) at various thresholds. ROC gives the trade-off between correctly identifying positive cases and incorrectly classifying negative cases as positive. The Area Under Curve of ROC provides a measure summarizing the model's overall performance with values ranging from 0.5 to 1.0. The value less than 0.5 indicates the miserable conditions of the trained model. It is worse than the random guess. A well performed model's curve approaches the top-left corner of the plot that indicates the high values of the high TPR and low FPR. The curve is used for comparing the different models, selecting optimal thresholds and analyzing model's performance in various operating points. It is important to note that the ROC curve is less informative for imbalanced datasets and does not indicate the model's probability calibration.

5) Evaluation By PR Curve

It is a graphical tool to evaluate the performance of the classification models, especially in the case of imbalanced datasets. It basically plots the precision against the recall under different thresholds. This curve gives insights on trade-off between precision and recall which is useful when FP and FN have different costs. Best model approaches the top-right corner of the plot. Unlike the ROC curve, the Precision-Recall curve's baseline depends on the class distribution in the dataset. It helps in selecting optimal thresholds and provides a detail information of model performance in imbalanced datasets compared to the ROC curve.

IV. RESULTS

The results obtained on applying the above described methodology are as follows:

The countplot for showing the count of different labels is shown in figure-(2) which shows that data for both labels are almost equal but count of stressed data is slightly more by about 100-200 entries.

The bar-plots for stress distribution based on the time-frames i.e. months, month-days and week-days are shown in figures-(3), (4) and (5) respectively and the same distribution for subreddit column is shown in figure-(6) which shows the almost

same distribution of stress and non stressed posts.

The word cloud for texts labelled as stress and no-stress are shown in figures-(7) and (8) respectively. It shows that the words like know, time are being used very often in both types of posts whereas the words like time, will are being frequently used in non-stressed posts whereas the words like feel, even, want etc are used in the stressed posts.

The heatmaps for different combination of Inflection removal techniques, word embedding and Naive Bayes types are shown from figure-(9) to (26). For heatmaps the x-axis shows the predicted and the y-axis shows the actual labels. Similarly, the ROC curves for different combination of Inflection removal techniques, word embedding and Naive Bayes types are shown from figure-(27) to (44).

The overall accuracy obtained when using these different combinations can be visualized in summary by the table-(3). These data show that the best techniques and models are: count-vectorization for word embedding, stemming/lemmatization for inflection reduction and Multinomial Naive Bayes.

Also the graph obtained from plotting the accuracy of model with respect to different α -parameters of Multinomial Naive Bayes is shown in figure-(45) which shows highest accuracy when $\alpha = 0.6$ or 0.7 . The heatmap, ROC curve and PR curve of the model when the α value was set to 0.7 are shown in figures-(46), (47) and (48) respectively.

V. DISCUSSION AND ANALYSIS

From the above analysis of the result, it can be seen that the Gaussian Naive Bayes is giving less accuracy among the other types of Naive Bayes. It may be due to the fact that the Gaussian Naive Bayes assume that the features follow a Normal Distribution but the textual and the categorical data in our dataset does not follow Normal distribution that has led to the poor performance of the Gaussian Naive Bayes. Also it may be due to having categorical data that has been encoded numerically that has led to information loss and distortions in the data distributions. And most importantly numerical, categorical and textual data had different distributions and naive bayes did not model these distributions effectively within the single framework.

But from the result it can be analyzed that the Multinomial Naive Bayes is giving higher accuracy than the other type of Naive Bayes. It is because multinomial Naive Bayes is especially designed for discrete data such as word counts in text analysis. It handles textual data more effectively than Gaussian Naive Bayes. Multinomial can handle categorical data without encoding too that preserves the inherent structure of the data. Multinomial Naive Bayes assumes that features are multinomially distributed, which is more appropriate for count-based data.

Hybrid Naive Bayes should have given the best accuracy among the other two types of Naive Bayes but from our result it is not so. It may be due to the Imbalance in the dataset i.e there are more columns for the textual data than the categorical and the numerical data. It caused the textual to dominate the model thereby reducing the effectiveness of the hybrid approach. Also, since the hybrid model is too complex hence it might have overfit the training data that has led to poor generalization for the unseen data.

In terms of the vectorization, hashing vectorization is giving less accuracy than the other vectorization techniques like Count Vectorization and TF-IDF techniques, looking with respect to the Multinomial Naive Bayes. Hashing vectorization has led to collisions where different terms are indexed to the same value, causing loss of information, semantic information and making the resulting vectors less informative. Unlike count vectorization and TF-IDF, hashing does not provide inherent term frequencies that are crucial for capturing the importance of terms in the textual data. Also the fixed length nature of hashed vectors may have lead to suboptimal representation for the dataset complexity.

In our analysis, Count Vectorization has shown higher accuracy compared to TF-IDF Vectorization. This progress is opposite of expectations, as TF-IDF should be able to represent textual data in a more subtle way. In the dataset when term frequency by itself is a very strong factor, then the simple counting of terms done by Count Vectorization may have been turned very effective. Also, if

the key terms in our dataset are equally important in all documents, TF-IDF might down-weight these important terms at the expense of its effectiveness. Further, the Multinomial Naive Bayes model has assumptions of feature independence and works effectively with count-based features, which does not optimally align with adjusted frequencies in TF-IDF.

Since the dataset is very small and terms don't vary much within it, then TF-IDF's emphasis on rare terms may not provide advantage over the simplicity of Count Vectorization. Hence, characteristic features in the dataset makes direct count vectorization yield better results than TF-IDF vectorization in the Multinomial Naive Bayes model for the dataset.

In this dataset, both stemmer and lemmatizer showed almost the same accuracy. It may be due to the nature of the text and the classification task at hand. Both reduced the words to its corresponding base/root form; it would reduce variations and hence probably the dimensionality of the feature space. The probable reason for the distinction at the output of the two processes in the dataset not highly affecting the classification accuracies may be that the context or grammatical changes of words may not carry any critical information about the classification task in the dataset and in the model training.

On implementation of the Multinomial Naive Bayes model, we found that the highest accuracy in prediction was with a value of alpha equal to 0.7. This fact was actually determined by grid search cross-validation. This alpha value balances in smoothing, and capturing the underlying distribution of the dataset. It returned an AUC value of 0.75 from ROC analysis. This score indicated that the model is able to distinguish between the different classes in the dataset. Also this value of alpha has effectively balanced bias and variance. Hence, this robustness has helped to capture the dataset's underlying patterns more accurately, thus giving better predictions on new data.

From the result it can be analyzed that the precision-recall curve indicates that the Multinomial Naive Bayes classifier had performed quite well. The AUC value of 0.81 indicates that the

model has perform much better than random guessing. The curve shows a characteristic downward trend as recall increases, but maintains high precision (above 0.8) for a significant portion of recall values. This gradual decline shows stable performance across various classification thresholds. The classifier balances precision and recall well for most of the thresholds, indicating reliable predictions. However, the sharp drop in precision at very high recall levels shows a common trade-off: capturing almost all positive instances comes at the cost of increased false positives.

VI. CONCLUSION

From the results obtained from the different inflection removal techniques, word embedding techniques and Naive Bayes types, it can be concluded that for working with the textual data, Multinomial Naive bayes works best whereas the Gaussian Naive Bayes performs worst. So, Gaussian Naive Bayes should only be deployed for prediction if the data consists of the numeric data with normal distribution. Also, if any semantic meaning is not to be used while prediction by the model, stemming and lemmatization do not make significant difference. So, to reduce the processing power, the stemming should be preferred in this case as it removes inflection based on fixed rules and consumes very less processing power. When using the word embedding techniques, if feature reduction is the only motive, hashing is generally preferred but in case of textual data it performs worst as it groups random words to random index without finding any specific relationship between the words hence reducing accuracy significantly. The TF-IDF and count vectorization methods both have comparatively equal effectiveness dealing with the textual data vectorization. The TF-IDF vectorization, in most of the cases performs well but in case where the word holds important meaning in context of classification (i.e repeated in almost all documents of same class and only in some documents of other class), the TF-IDF reduces its importance due to its nature to give less importance in to the words that occur in most of documents hence supresses the important information for classification thus reducing the accuracy of the model. In such case,

count vectorization works the best. In this dataset, the final feature set exceeds the number of data-points by a multiplicative factor of 3 or 4 leading the model to perform less than expected. Thus, for textual data, to get more accuracy the model needs to be trained on abundance of data; much higher than those required for training other type of data.

APPENDIX A FLOW CHART

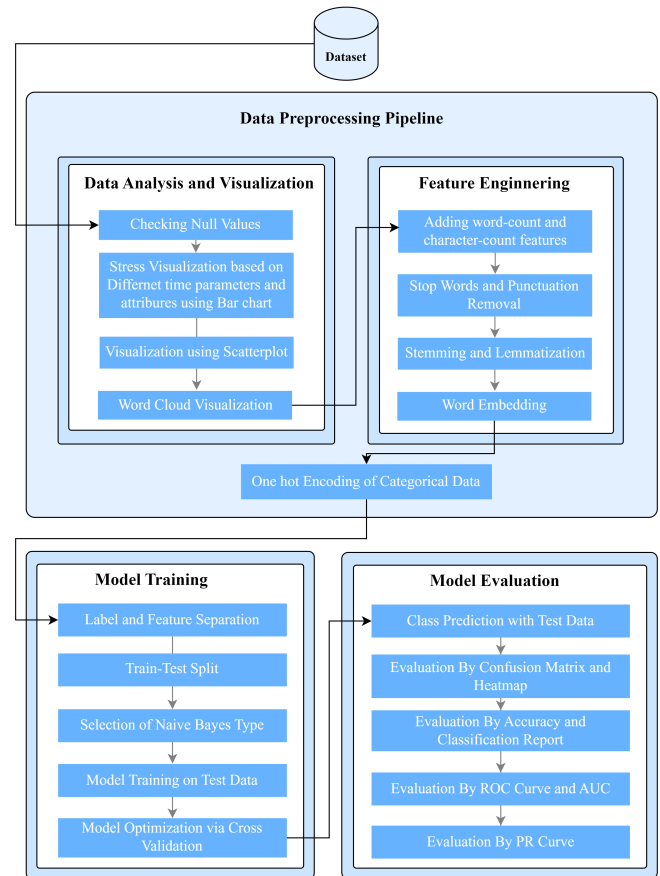


FIGURE 1. System Block Diagram

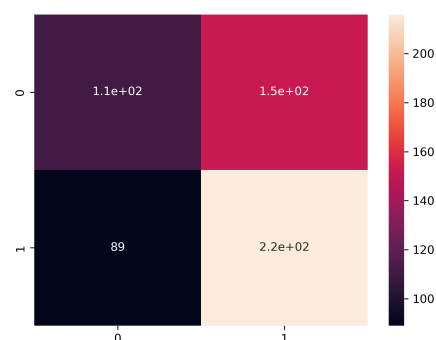
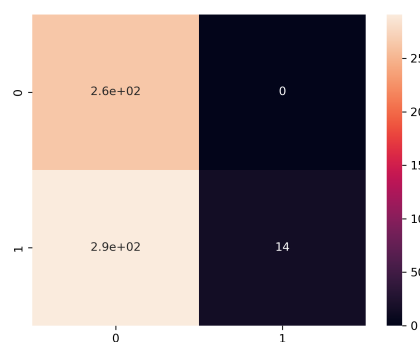


FIGURE 8. Word Cloud for Texts labelled as No-stress



	0	1
0	1.4e+02	1.2e+02
1	99	2.1e+02

	0	1
0	1.4e+02	1.2e+02
1	1e+02	2e+02

	0	1
0	1.7e+02	90
1	67	2.4e+02

VOLUME 1, 2024

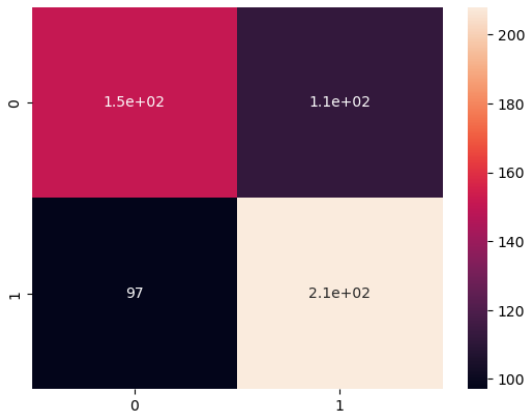


FIGURE 14. Heatmap when using Lemmatization, TF-IDF Vectorization and Hybrid Naive Bayes

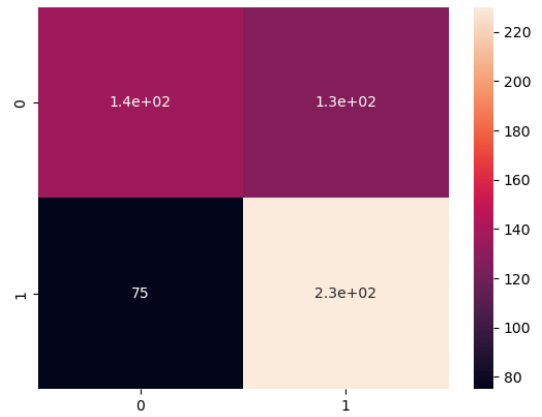


FIGURE 17. Heatmap when using Lemmatization, Count Vectorization and Hybrid Naive Bayes

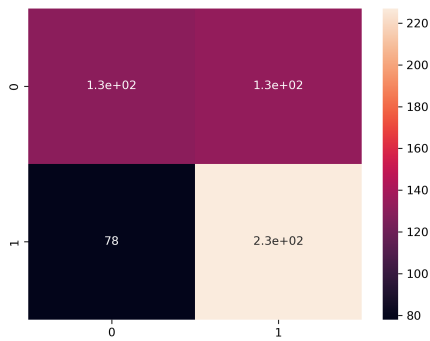


FIGURE 15. Heatmap when using Lemmatization, Count Vectorization and Gaussian Naive Bayes

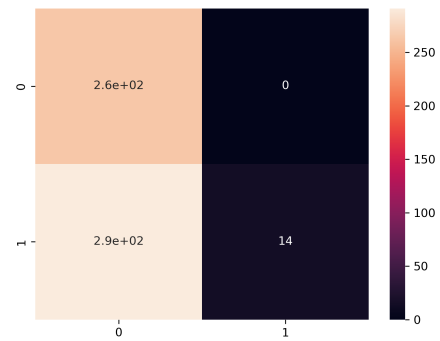


FIGURE 18. Heatmap when using Stemming, Hashing Vectorization and Gaussian Naive Bayes

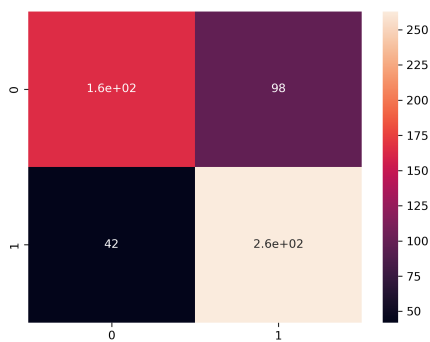


FIGURE 16. Heatmap when using Lemmatization, Count Vectorization and Multinomial Naive Bayes

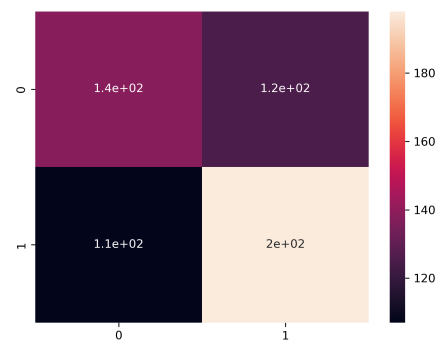


FIGURE 19. Heatmap when using Stemming, Hashing Vectorization and Multinomial Naive Bayes

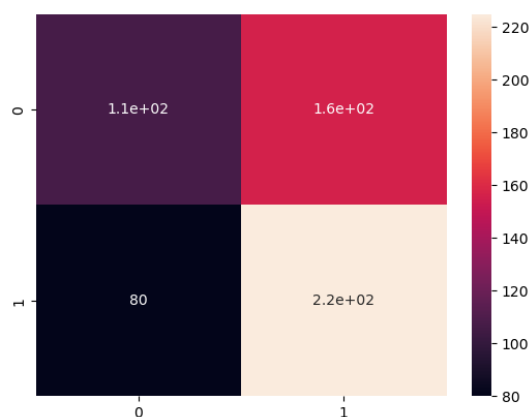


FIGURE 20. Heatmap when using Stemming, Hashing Vectorization and Hybrid Naive Bayes

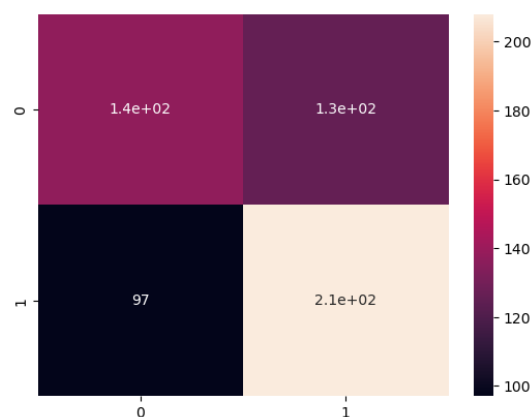


FIGURE 23. Heatmap when using Stemming, TF-IDF Vectorization and Hybrid Naive Bayes

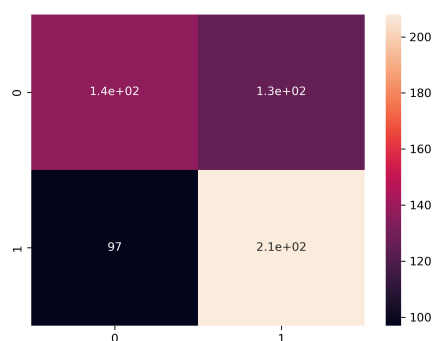


FIGURE 21. Heatmap when using Stemming, TF-IDF Vectorization and Gaussian Naive Bayes

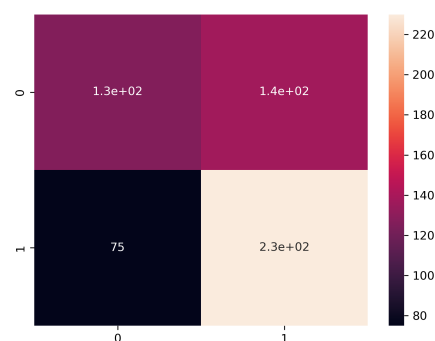


FIGURE 24. Heatmap when using Stemming, Count Vectorization and Gaussian Naive Bayes

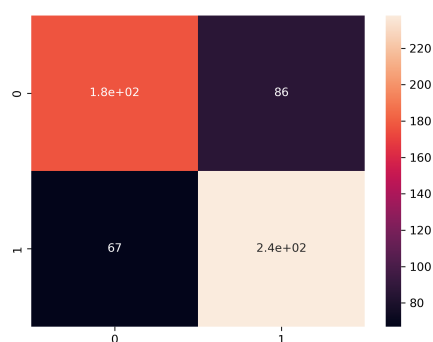


FIGURE 22. Heatmap when using Stemming, TF-IDF Vectorization and Multinomial Naive Bayes

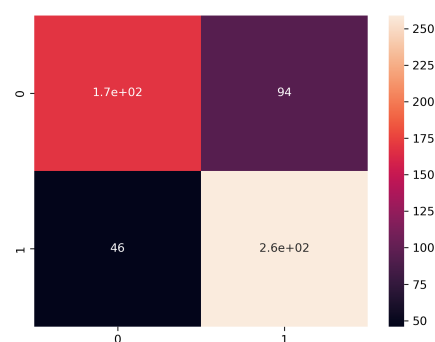


FIGURE 25. Heatmap when using Stemming, Count Vectorization and Multinomial Naive Bayes

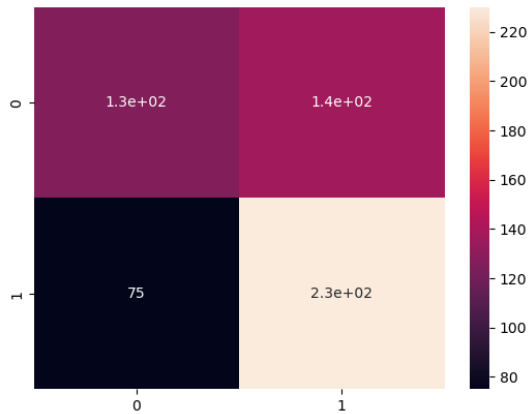


FIGURE 26. Heatmap when using Stemming, Count Vectorization and Hybrid Naive Bayes

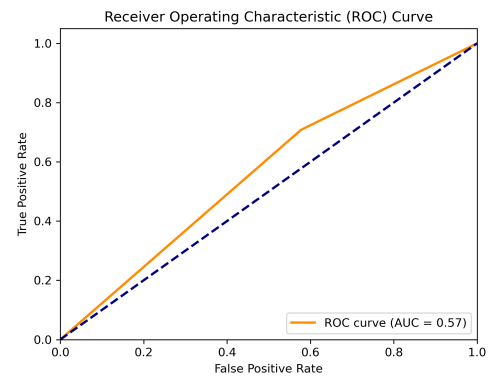


FIGURE 29. ROC Curve when using Lemmatization, Hashing Vectorization and Hybrid Naive Bayes

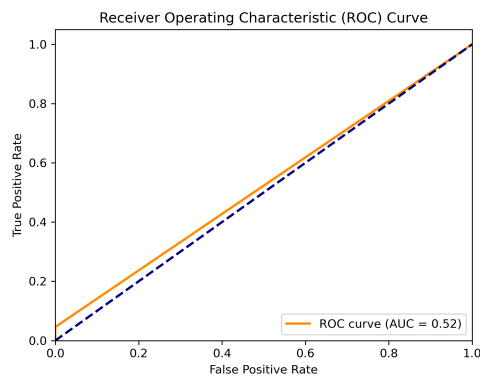


FIGURE 27. ROC Curve when using Lemmatization, Hashing Vectorization and Gaussian Naive Bayes

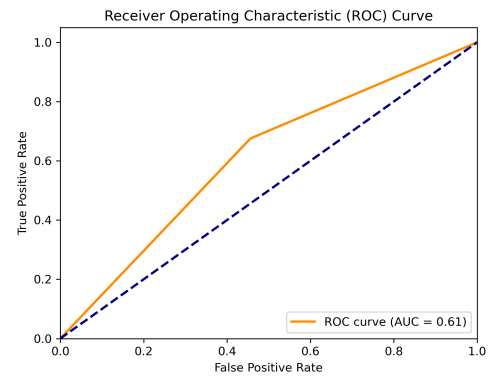


FIGURE 30. ROC Curve when using Lemmatization, TF-IDF Vectorization and Gaussian Naive Bayes

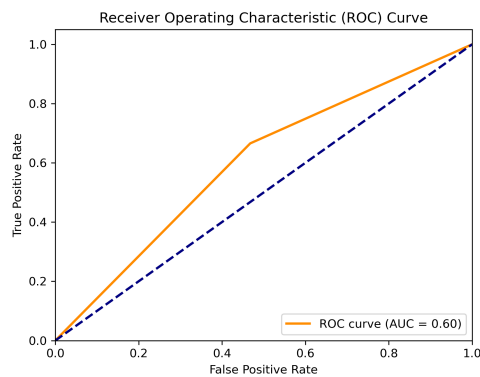


FIGURE 28. ROC Curve when using Lemmatization, Hashing Vectorization and Multinomial Naive Bayes

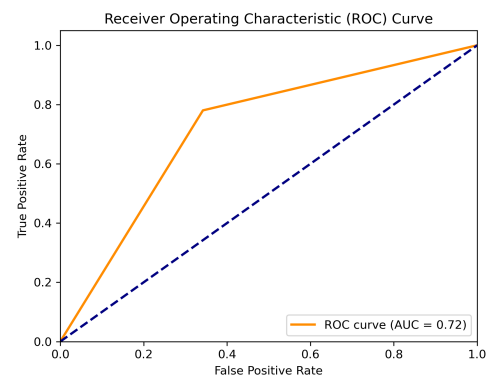


FIGURE 31. ROC Curve when using Lemmatization, TF-IDF Vectorization and Multinomial Naive Bayes

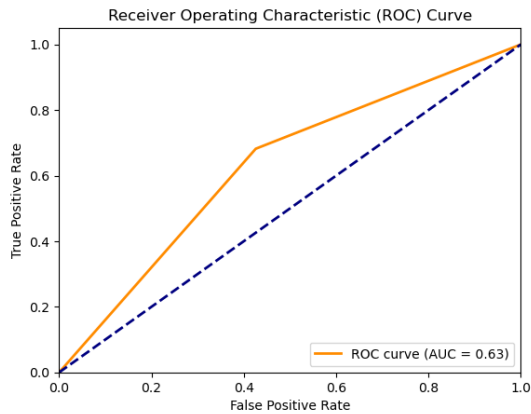


FIGURE 32. ROC Curve when using Lemmatization, TF-IDF Vectorization and Hybrid Naive Bayes

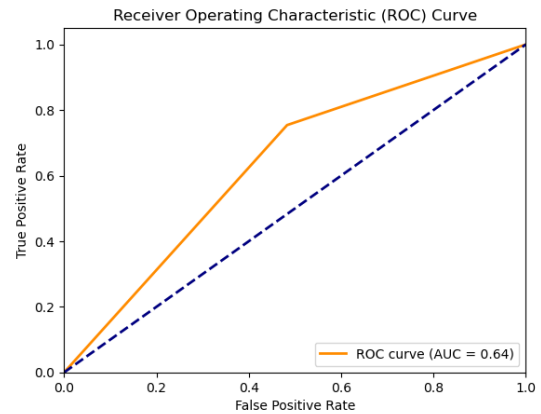


FIGURE 35. ROC Curve when using Lemmatization, Count Vectorization and Hybrid Naive Bayes

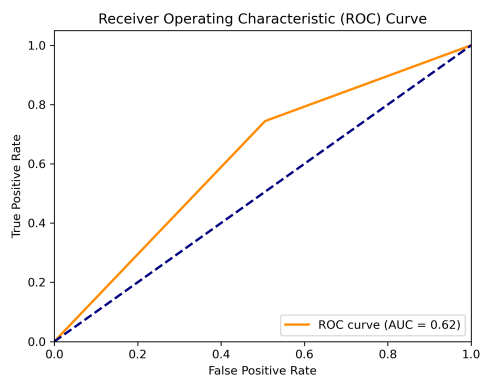


FIGURE 33. ROC Curve when using Lemmatization, Count Vectorization and Gaussian Naive Bayes

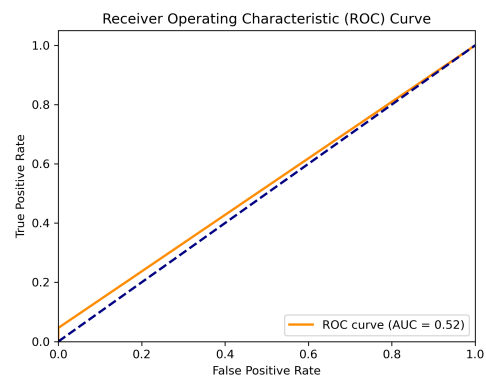


FIGURE 36. ROC Curve when using Stemming, Hashing Vectorization and Gaussian Naive Bayes

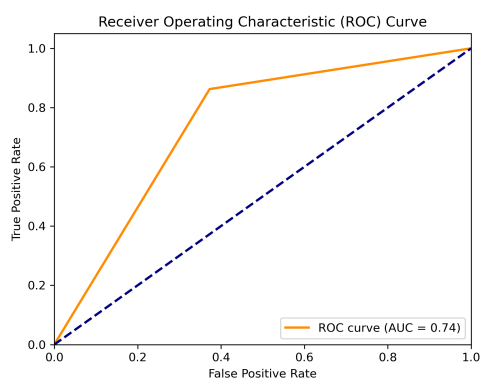


FIGURE 34. ROC Curve when using Lemmatization, Count Vectorization and Multinomial Naive Bayes

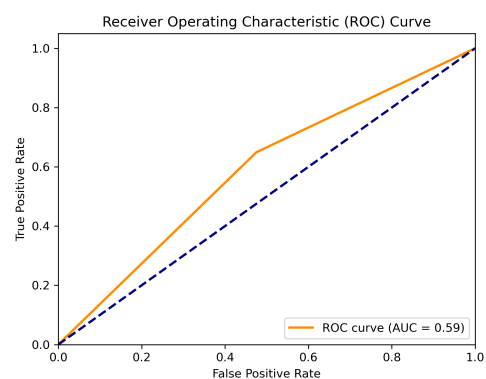


FIGURE 37. ROC Curve when using Stemming, Hashing Vectorization and Multinomial Naive Bayes

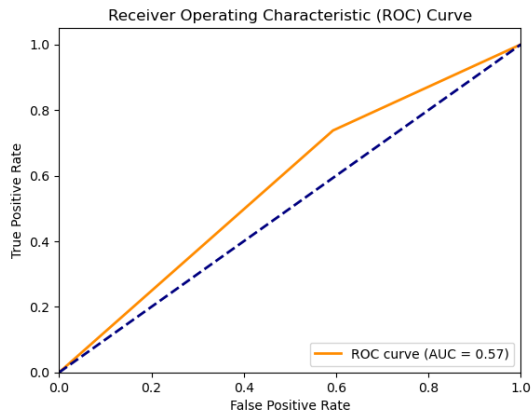


FIGURE 38. ROC Curve when using Stemming, Hashing Vectorization and Hybrid Naive Bayes

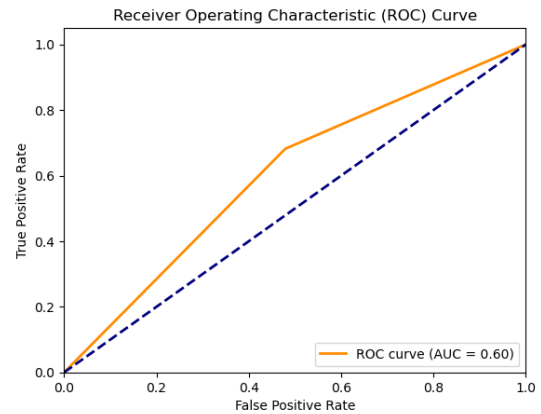


FIGURE 41. ROC Curve when using Stemming, TF-IDF Vectorization and Hybrid Naive Bayes

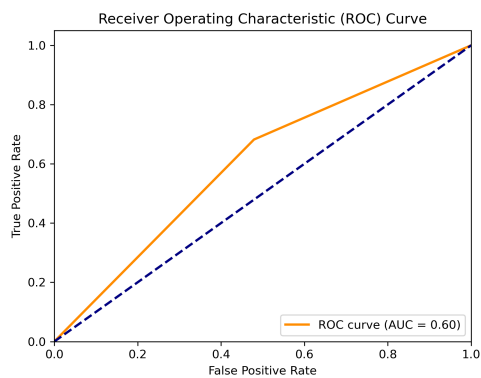


FIGURE 39. ROC Curve when using Stemming, TF-IDF Vectorization and Gaussian Naive Bayes

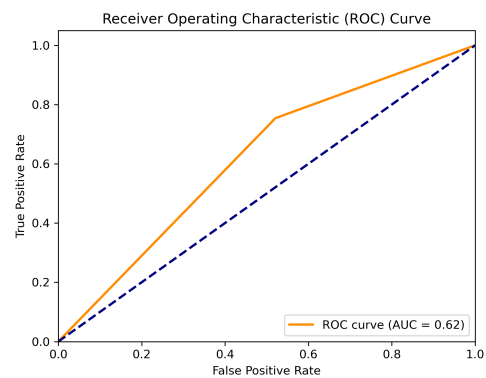


FIGURE 42. ROC Curve when using Stemming, Count Vectorization and Gaussian Naive Bayes

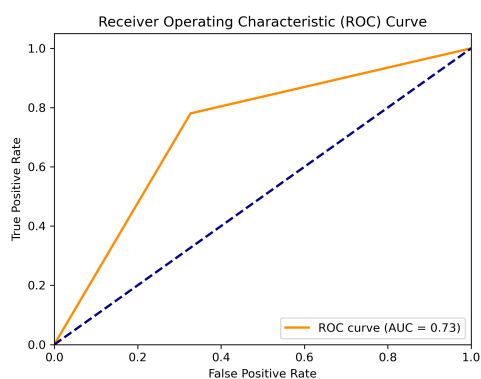


FIGURE 40. ROC Curve when using Stemming, TF-IDF Vectorization and Multinomial Naive Bayes

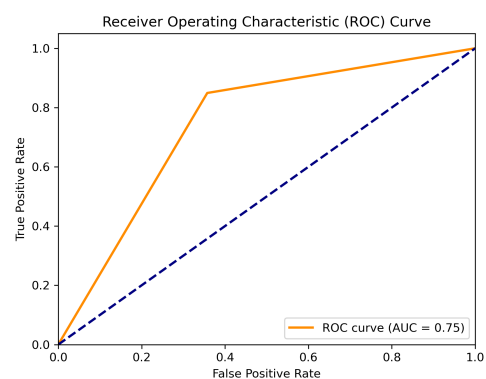


FIGURE 43. ROC Curve when using Stemming, Count Vectorization and Multinomial Naive Bayes

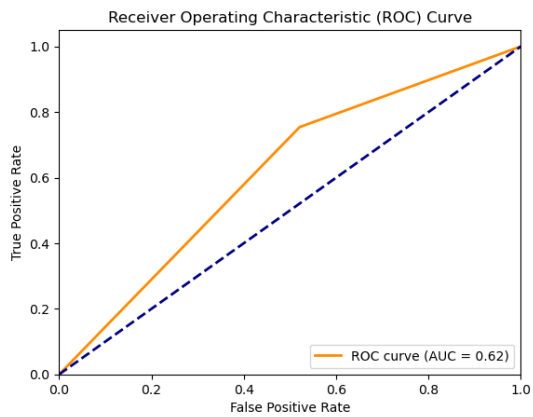


FIGURE 44. ROC Curve when using Stemming, Count Vectorization and Hybrid Naive Bayes

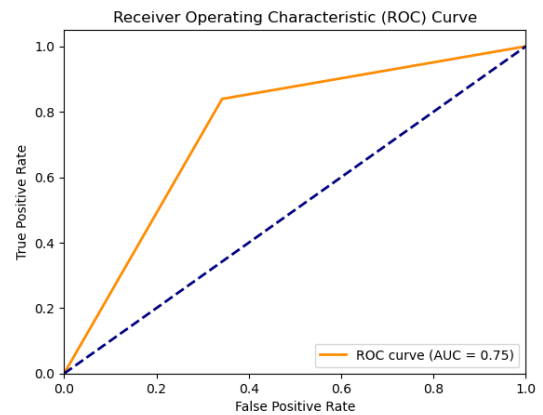


FIGURE 47. ROC Curve when using Stemming, Count Vectorization and Multinomial Naive Bayes with $\alpha = 0.7$

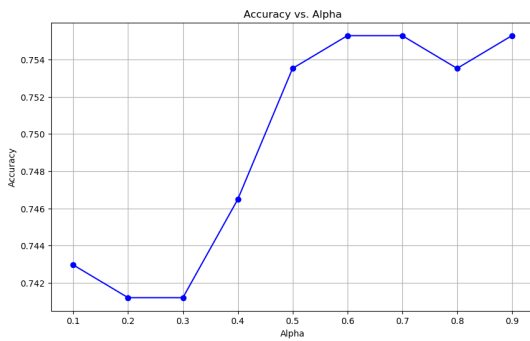


FIGURE 45. Accuracy based on α hyperparameter of Naive Bayes

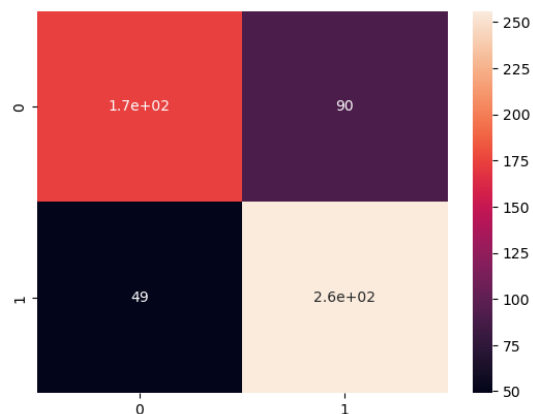


FIGURE 46. Heatmap when using Stemming, Count Vectorization and Multinomial Naive Bayes with $\alpha = 0.7$

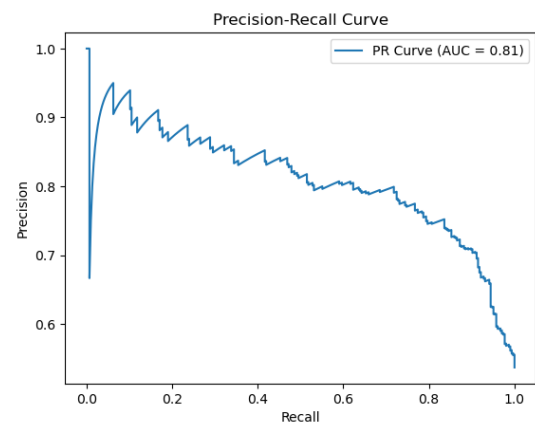


FIGURE 48. PR Curve when using Stemming, Count Vectorization and Multinomial Naive Bayes with $\alpha = 0.7$

APPENDIX C FORMULA EQUATIONS

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)} \quad (1)$$

where,

$P(A|B)$ is the posterior probability; the probability of (A) given (B) has occurred,

$P(B|A)$ is the likelihood; the probability of B given (A) has occurred,

$P(A)$ is the prior probability of (A); marginal probability of (A)

$P(B)$ is the marginal likelihood or evidence; marginal probability of (B)

p_i = Probability of an item belonging to class i,

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (2)$$

where,

$P(A \cap B)$ is the probability of both (A) and (B) occurring together,

$P(B)$ is the marginal probability of occurrence of (B)

p_i = Probability of an item belonging to class i,

$$P(X_1, X_2, \dots, X_n|C) = P(X_1|C) * P(X_2|C) * \dots * P(X_n|C) \quad (3)$$

$$TF(t, d) = \frac{n(t)}{N(d)} \quad (4)$$

where,

$n(t)$ = Number of times term t appears in document d

$N(d)$ = Total number of terms in document d

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right) \quad (5)$$

where,

N = total number of documents in the corpus

$DF(t)$ = number of documents containing the term t

$$TF - IDF(t, d, D) = TF(t, d) * IDF(t) \quad (6)$$

where,

t = term

d = document

D = corpus

$$P(Class|Features) = \frac{P(Features|Class) * P(Class)}{P(Features)} \quad (7)$$

where,

$P(Class|Features)$ is the posterior probability,

$P(Features|Class)$ is the likelihood,

$P(Class)$ is the prior probability,

$P(Features)$ is the evidence.

$$Class_{MAP} = \operatorname{argmax}_c P(C = c|X) = \operatorname{argmax}_c [P(X|C = c) * P(C = c)] \quad (8)$$

where,

C represents the class,

X represents the features,

$P(C=c|X)$ is the posterior probability,

$P(X|C=c)$ is the likelihood,

$P(C=c)$ is the prior probability.

$$P(feature|class) = \frac{count + \alpha}{total + \alpha * k} \quad (9)$$

Where,

'count' is the number of times the feature appeared with the class,

'total' is the total count of all features for the class,

α is the smoothing parameter (usually 1 for Laplace smoothing).

and k is number of distinct features within the class

APPENDIX D CODE SNIPPETS

```
1 df['MONTH'] = df['social_timestamp'].apply(
    lambda date: datetime.fromtimestamp(date)
    .month)
2 df['Day'] = df['social_timestamp'].apply(
    lambda date: datetime.fromtimestamp(date)
    .day)
3 df['WeekDay'] = df['social_timestamp'].apply
    (lambda date: datetime.fromtimestamp(date)
    .weekday())
```

Listing 1. Time Information Retrieval

```
1 df_viz_binning = pd.pivot_table(data =
    new_df, index=['social_timestamp'],
    values=['label', 'inv_label'], aggfunc =
    'sum')
2 df_viz_binning.rename(columns = {'label':'
    stress', 'inv_label':'no-stress'},
    inplace = True)
3 df_viz_months.plot.barh()
4
5 df_viz_days = pd.pivot_table(data = new_df,
    index =['Day'], values=['label', '
    inv_label'], aggfunc = 'sum')
6 df_viz_days.rename(columns = {'label':'
    stress', 'inv_label':'no-stress'},
    inplace = True)
7 df_viz_days.plot.barh()
8
9 df_viz_weekdays = pd.pivot_table(data =
    new_df, index =['WeekDay'], values=['
    label', 'inv_label'], aggfunc = 'sum')
10 df_viz_weekdays.rename(columns = {'label':'
    stress', 'inv_label':'no-stress'},
    inplace = True)
11 df_viz_weekdays.plot.barh()
```

Listing 2. Bar-Chart Showing Distribution of Stress Posts

```
1 from wordcloud import WordCloud, STOPWORDS
2
3 comment_words = ''
4 stopwords = set(STOPWORDS)
5
6 for val in (df.loc[df['label'] == 1]).text:
7
8     val = str(val)
9
10    tokens = val.split()
11
12    for i in range(len(tokens)):
13        tokens[i] = tokens[i].lower()
14
15    comment_words += " ".join(tokens)+" "
16
17 wordcloud = WordCloud(width = 800, height =
    800,
18
19        background_color = 'white',
20        stopwords = stopwords,
21        min_font_size = 10).generate
    (comment_words)
22
23 plt.figure(figsize = (8, 8), facecolor =
    None)
24 plt.imshow(wordcloud)
```

```
24 plt.axis("off")
25 plt.tight_layout(pad = 0)
26
27 plt.show()
```

Listing 3. Code For Word-cloud Visualization

```
1 import string
2 punctuations = string.punctuation
3 texts = []
4 for text in df.text:
5     text = text.lower()
6     tokens = []
7     for word in text.split():
8         if not (word in stopwords):
9             chars = []
10            for char in word:
11                if not (char in punctuations)
12                :
13                    chars.append(char)
14                    tokens.append(''.join(chars))
15 texts.append(tokens)
16 df.text = texts
```

Listing 4. Code For Stopwords and Punctuation Removal

APPENDIX E TABLES

TABLE 1. Missing Values in the Dataset

Column	Missing Values
Subreddit	0
Post ID	0
Sentence Range	0
Text	0
Label	0
Confidence	0
Social Timestamp	0

TABLE 2. Classification Report of Weather Dataset with Entropy as Splitting Criterion and No max-depth provided

	Precision	Recall	F1-Score	Support
No	0.50	1.00	0.67	1
Yes	1.00	0.75	0.86	4
Accuracy			0.80	5
Macro avg	0.75	0.88	0.76	5
Weighted avg	0.90	0.80	0.82	5

TABLE 3. Comparison of Naive Bayes Types with Different Inflection Removal and Word Embedding Techniques

Inflection Removal Technique	Word Embedding Technique	NB Type	Accuracy
Lemmatization	Hashing	Gaussian	49
		Multinomial	60
		Hybrid	58
	TF-IDF	Gaussian	61
		Multinomial	72
		Hybrid	63
Stemming	Count	Gaussian	63
		Multinomial	75
		Hybrid	64
	Hashing	Gaussian	49
		Multinomial	59
		Hybrid	58
	TF-IDF	Gaussian	61
		Multinomial	73
		Hybrid	61
	Count	Gaussian	63
		Multinomial	75
		Hybrid	63

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Dinesh Kshetri Baniya (DBK sir) for his invaluable guidance and support throughout this lab exercise on Naive Bayes Classification at IOE Thapathali Campus. His insightful feedback and encouragement have greatly enriched our understanding of the subject matter and enhanced the quality of this report. DBK sir has inspired us to undertake deep analysis of the topic, which has helped us gain valuable insights on every topic we explore.

We are also thankful to our classmates and peers for their collaboration and constructive discussions during the lab sessions. Their diverse perspectives and inputs have contributed significantly to the depth and breadth of our learning experience.

Additionally, we would like to acknowledge Department of Electronics and Computer Engineering, IOE, Thapathali Campus for providing the necessary resources and facilities that enabled us to conduct this research effectively.

Last but not least, we extend our appreciation to our families and friends for their unwavering encouragement and understanding during this academic endeavor.

REFERENCES

- [1] P. Domingos, "Tapping into the 'folk knowledge' needed to advance machine learning applications," *Communications of the ACM*, vol. 55, doi: 10.1145/2347736.2347755, 2012.
- [2] Chingmuankim and R. Jindal, "Classification and Analysis of Textual data using Naive Bayes with TF-IDF," 2022 4th International Conference on Electrical, Control and Instrumentation Engineering (ICECIE), Kuala Lumpur, Malaysia, 2022, pp. 1-9, doi: 10.1109/ICECIE55199.2022.10000309.
- [3] Shaikh and R. Patil, "Fake News Detection using Machine Learning," 2020 IEEE International Symposium on Sustainable Energy, Signal Processing and Cyber Security (iSSSC), Gunupur Odisha, India, 2020, pp. 1-5, doi: 10.1109/iSSSC50941.2020.9358890.
- [4] P. N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, "Introduction to Data Mining," India: Pearson, 2009.
- [5] DataCamp, "Stemming and lemmatization in Python," DataCamp, [Online]. Available: <https://www.datacamp.com/tutorial/stemming-lemmatization-python>
- [6] T. Datascience, "Different techniques to represent words as vectors: Word embeddings,"

Towards Data Science, [Online]. Available:
<https://towardsdatascience.com/different-techniques-to-represent-words-as-vectors-word-embeddings-3e4b9ab7ceb4>



RIJAN POKHREL is currently pursuing a Bachelor's Degree in Electronics and Information Engineering at IOE Thapathali Campus, as of 2024. His passion lies in leveraging AI to revolutionize industries and solve complex problems. He is a motivated learner, constantly staying updated with the latest advancements in AI. Currently exploring quantum machine learning and proficient in app development using Flutter and backend development with Firebase.

Passionate about mathematics, statistics, and machine learning, R. Pokhrel possesses a keen interest in discovering hidden insights from data. He aspires to bridge the financial sector with automated technology through artificial intelligence. Additionally, he is a strong advocate for free speech and is deeply interested in politics and the stock market.



SABIN ACHARYA is an undergraduate student studying Electronics, Communication and Information Engineering in Institute of Engineering, Thapathali Campus, Tribhuvan University graduating in April 2025.

He is a Tech-enthusiast with keen interests in circuits and electronics along with Machine Learning and Software Development. He has skills like Android Application Development with Flutter, Backend with Node.js and Machine Learning Skills with Python. His recent projects include "Virtual Simulation of Hand Gestures Using Hall Effect Sensors". Apart from academics, he is keenly interested in portrait sketching and music.

Mr. Acharya was the recipient of Semester Topper Award from Thapathali Campus in the respective fields for 4 times till March 2024.

...